Analysis and Integration of Optimization Solvers for JalTantra

Submitted in partial fulfillment of
the requirements for the degree of
Master of Technology
by

Fenil Gaurang Mehta
(203050054)

Supervisor:
Prof. Om Damani

Department of Computer Science and Engineering
Indian Institute of Technology Bombay
Mumbai 400076 (India)

June 2022

Dedicated to my beloved family

Acceptance Certificate

Department of Computer Science and Engineering
Indian Institute of Technology, Bombay

The thesis entitled “Analysis and Integration of Optimization Solvers for JalTantra” submitted by Fenil Gaurang Mehta (203050054) may be accepted for being evaluated.

Date: June 2022

Prof. Om Damani

Approval Sheet

This thesis entitled “Analysis and Integration of Optimization Solvers for JalTantra” by Fenil Gaurang Mehta is approved for the degree of Master of Technology.

_____

_____

_____

Examiners

_____

_____

_____

Supervisor (s)

_____

Chairman

Date: _____

Place: _____

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I declare that I have properly and accurately acknowledged all sources used in the production of this report. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Fenil Gaurang Mehta

Date: 24 June 2022 (203050054)

Abstract

Water is a necessity for living, and to get water at our homes water distribution pipeline networks need to be established. The designing of this pipeline network is a big challenge in itself because various factors come into play. For example, the pipeline establishment cost needs to be minimized, the demand at each location of the network needs to be satisfied, the designers also have to keep a check on the energy consumptions of the pumps or other machinery that would be used in maintaining the water supply, etc. It is not possible to manually keep check of all the factors that affect the resulting system, and configure them to have the resulting system efficient as well. This is where water network design systems like JalTantra come into picture.

JalTantra is a water network design system that optimizes pipeline network establishment cost while at the same time ensuring that the demand and pressure constraints at each point in the network are satisfied. Initially JalTantra used to only support branched networks (i.e. a tree-like network), and when I had begun working on the project it was somewhat able to optimize small cyclic networks with the help of COIN-OR Branch and Cut (CBC) solver [1]. But, it was noticed that converting this water pipeline network problem to general nonlinear constrained-optimization problem and using global optimizers gave better results in less time [2].

Analyzing global optimization solvers with different configurations for nonlinear programming formulation, and integrating them with JalTantra was my contribution to this project and I explain the details about the same in this thesis.

Table of Contents

List of Figures

List of Tables

List of Abbreviations

CBC	COIN-OR Branch and Cut
CLI	Command Line Interface
CPU	Central Processing Unit
ILP	Integer Linear Programming
IP	Internet Protocol
JSON	JavaScript Object Notation
JSP	JavaServer Pages
MINLP	Mixed Integer Nonlinear Programming
NLP	Nonlinear Programming

Chapter 1

Introduction

JalTantra is a water network design system that optimizes pipeline network establishment cost while at the same time ensuring that the demand and pressure constraints at each point in the network are satisfied. It has a very intuitive user interface that makes the water network designing process very simple and easy.

Previously, work has been done to map the water pipeline network optimization problem to integer linear programming (ILP) [1] and nonlinear programming (NLP) problems [2]. My work will build on it. To get more details regarding the concepts and terminology used in this thesis, please refer [3], [1], [2].

1.1 Problem Statement

The integer linear programming (ILP) formulation was integrated into JalTantra to solve cyclic networks with the help of COIN-OR Branch, and Cut (CBC) solver [1]. However, it was quite limited in its capabilities regarding the size of the network that it can handle. On the other hand, the nonlinear programming (NLP) problem formulation was not integrated.

The ILP formulation could only handle small networks like the two loop network (Figure 1.1), and when a network with about two hundred nodes was submitted, it did not return any result even after hours. This is where the NLP formulation of the water pipeline network optimization problem comes into picture. It was able to solve the two hundred node network within a few minutes. But, there are various configurable options when working with the NLP formulation. "Chapter 2" describes the details of the analysis that was done to decide which configuration to use, and "Chapter 3" describes the details regarding the integration of "the solvers that can solve the NLP formulation of the pipeline network optimization problem" and "the JalTantra frontend".

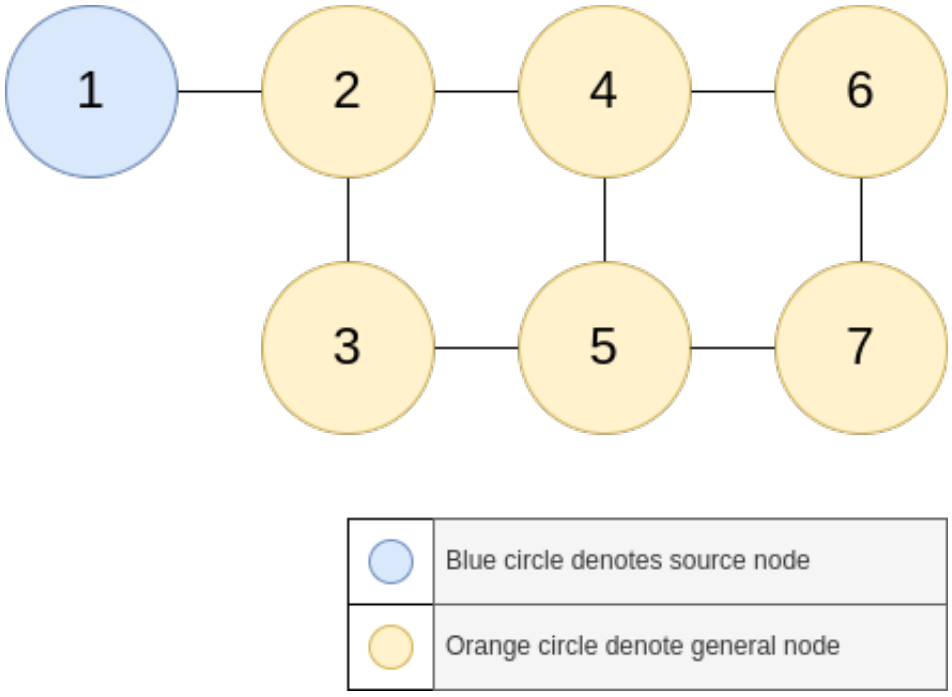

Figure 1.1: Two Loop Network

Chapter 2

Solvers and Models

2.1 Introduction to Solvers and Models

Solver is a software which can be in the form of a standalone computer program/binary or as a programming library, that can solve mathematical problems [4]. They are to be given the problem description in a standard format and it would find the solution.

We began with choosing Octeract as the solver that we would test and evaluate for its usefulness for our work because it was known to us. **Octeract** is a proprietary massively parallel deterministic global optimization solver for general Mixed-Integer Nonlinear Programs (MINLP) [5]. And, to use Octeract, we used a software called AMPL. **AMPL** is a comprehensive and powerful algebraic modeling language for linear and nonlinear optimization problems, in discrete or continuous variables. Developed at Bell Laboratories, AMPL lets us use common notation and familiar concepts to formulate optimization models and examine solutions, while the computer manages communication with an appropriate solver [6].

The first thing that I did was to install AMPL and Octeract, and try them on the model and data file already available with us. The **model file** declares the data parameters, the variables, objective function, and the constraints in a symbolic fashion [7]. All the numbers (i.e. the actual data) are provided in the **data file** [7]. We can consider solver to be analogous to function definition, model file to be equivalent to function declaration, and data file to be equivalent to the data/parameters that we pass when calling the function.
Following is the list of all solvers that we tested:

- Octeract
- Baron
- Gurobi
- CPLEX

- Xpress
- HiGHS

Before using the solver for performance evaluation, we tested them manually on small network files and following is the conclusion that we drew:

- **Octeract** worked correctly and it was able to solve the network files we gave it because it is a global MINLP solver. A paid license of one year for this was bought.
- **Baron** too worked great as it is a general nonlinear optimizer capable of solving nonconvex optimization problems. This was directly available with AMPL's paid license. We did not buy this or do anything special to use this, it was already present in the installation directory of "ampl".
- Gurobi could not solve the network files used for testing because
  - It can only solve integer, mixed integer and linear programming problems
  - It can not handle nonquadratic nonlinear constraints of JalTantra network files
- CPLEX and Xpress, both did not solve the network files used for testing because
  - They only supports problem types: Linear and quadratic optimization in continuous and integer variables
- HiGHS could not be used for the network files during testing because it is not directly compatible with AMPL, and after investing a week's time it was found that it is "not possible" to convert our current model and data input format to that required by HiGHS.

Due to time constraints, it was decided that we should proceed with Octeract and Baron only because spending more time on searching more solvers could delay the development work of integrating these solvers with the JalTantra website, and thus delaying the deployment of the working system.

2.2 Solver and Model Analysis - Technique

To decide upon which solver-model combinations to be used for production environment (i.e. deployment), we used all combinations of available solvers, model files and data files for our analysis.
Solvers used:

1. Octeract

2. Baron

The solvers also have the power to use multiple **CPU cores**. So, we did testing for three cases:

1. One CPU core
2. Four CPU cores
3. Sixteen CPU cores

We had four different **model files** available with us, namely:

1. m1_basic.R
2. m2_basic2.R, m2_basic2_v2.R
    - While testing the solvers and models we also found that solvers were supposedly able to find the global optimal solution for model "m2_basic2.R" for the data files which we used for testing. However, this global optimal was out performed by the model "m1_basic.R" for some data files. After vetting the model "m2_basic2.R", it was found that it had made wrong assumptions (that (a) two parallel pipes would be installed between two points, and (b) when flow of water is in one direction according to the results of the optimization of the network then some non-zero amount of water would flow in the opposite direction as well) about the real life pipeline establishment which had resulted in incorrect constraints in the model file. The fixed version of model "m2_basic2.R" was named "m2_basic2_v2.R" and this fixed version was used for all the work after that point of time.
3. m3_descrete_segment.R
4. m4_parallel_links.R

We used eleven **data files**, namely:

1. d1_Sample_input_cycle_twoloop.dat
2. d2_Sample_input_cycle_hanoi.dat
3. d3_Sample_input_double_hanoi.dat
4. d4_Sample_input_triple_hanoi.dat
5. d5_Taichung_input.dat
6. d6_HG_SP_1_4.dat

7. d7_HG_SP_2_3.dat
8. d8_HG_SP_3_4.dat
9. d9_HG_SP_4_2.dat
10. d10_HG_SP_5_5.dat
11. d11_HG_SP_6_3.dat

Total combinations = $(2 * 3) * (4 * 11) = 6 * 44 = 264$

For all these combinations, we made use of a 48 core server, and it was decided by us that we would have the maximum time limit for each combination as four hours.

Following challenges came up when these combinations had to be executed for analysis:

1. Stopping the solvers as soon as the four hour time limit is reached
2. Making effective use of the server 24/7 and ensuring that no time is wasted in switching from execution of one combination to another.
3. Avoiding overloading of the CPU (e.g. launching four combinations where each combination is using 16 CPU cores would mean that we are committing 64 CPU cores while only 48 are available – this leads to contention among the processes for CPU resources)
4. Managing the results of all the combinations
5. Ensuring that sufficient amount of free RAM is available for all running instances of the combinations.

It was not possible to take care of the above-mentioned challenges manually. So, a Python program was written to resolve all the challenges (available at private GitHub repository [8]). The combinations were split into six groups (based on the solver and the CPU cores they could use)

1. Octeract, 1 CPU core
2. Octeract, 4 CPU cores
3. Octeract, 16 CPU cores
4. Baron, 1 CPU core
5. Baron, 4 CPU cores
6. Baron, 16 CPU cores

This program managed all the things automatically in the following manner:

1. The program would continuously monitor each running instance of the solver and would stop them once they cross the four hour time limit.
2. To make proper use of the server, the program would launch at most "48/*number_of_CPU_cores_configured_for_that_batch*" instances of the solver at a time.
3. To avoid CPU overload, the program would launch a new instance of solver only when less than "48/*number_of_CPU_cores_configured_for_that_batch*" solver instances are running.
4. The results were stored in files using a consistent naming convention.
5. The program would continuously monitor the amount of free RAM in the system, and if the free RAM goes below a certain threshold, then it would stop the solver instance which has been running for the maximum amount of time among the running instances.

The summary of the results of these 264 combinations is present in Appendix A. And, the details have been put in a spreadsheet available at [9]. Organization of data in this spreadsheet is in the following manner:

1. There is a separate sheet for each entity of the grouping explained above (i.e. combination of Solver and CPU cores used). Following are their names:
   (a) "Octeract - 1 cores, 4hours"
   (b) "Octeract - 4 cores, 4hours"
   (c) "Octeract - 16 cores, 4hours"
   (d) "Baron - 1 cores, 4hours"
   (e) "Baron - 4 cores, 4hours"
   (f) "Baron - 16 cores, 4hours"
2. There is one primary sheet (named "Quick Summary Generator") which is capable of extracting the necessary data from the above-mentioned sheet and dynamically generating a summary table. Conditional formatting has been done on this summary table to highlight important information/values in it, and make it easy to analyze and draw conclusions from the same.

The spreadsheet has been designed in such a way that it can be reused in future if more data related to any other solver is added, provided it is inserted in new sheets according to the already existing format. For example, let us consider that we want to add data related to a solver named ABC which was configured to use only 4 CPU cores. Follow the below steps:

1. Create a copy of an already existing sheet. Let us say, we use "Octeract - 16 cores, 4hours" or "Baron - 1 cores, 4hours".

2. Rename the sheet to "ABC - 4 cores, 4hours" (Do not add or remove any space or any other letter from the sheet name)

3. Ensure that first four rows of column A and B are filled correctly using the new solver name and CPU core limit

4. Columns D, E and columns after them can be used to add any metadata related to the execution

5. Just update the data in the table A6:L104.

    - Do not increase or decrease the number of rows/columns under each model-data combination.
    - Only eight rows and three columns should be present under each table of model-data combination.
    - Neither change the name (i.e. "m1_d1.txt", "m2_d1.txt", etc) of any of the model-data table, nor change their order

To get the values from this newly created sheet into the summary table, just write "4", "ABC", "0:05:00" (or any other value of time in "hh:mm:ss" format, like "4:00:00" for four hours) in row 11,12,13 respectively under the column "just next to the last column of the summary table".

2.3 Solver and Model Analysis - Conclusions

- The conclusions are primarily for model m1 and m2, because
    - Model m3 (with both solvers Octeract and Baron) did not work on data files d5, d6, d7, d8, d9, d10 and d11 (i.e. $\approx 63\%$ of the data files)
    - Model m3 and m4 require "cycles in the graph" to be present in the data file. So, from the perspective of integration of solvers and models with the JalTantra website, it would vastly increase the scope of the project and would put the possibility of "end

to end integration" at risk because we would have to work on detecting cycles in the requests submitted by the user from the frontend. If this is going to be implemented in future, then there is one research paper that could prove helpful as it discusses finding loops in water distribution networks [10].

- Execution time limit of 5 minutes
  - Model m2 always performed better than m1 for "Octeract - 1 core"
  - Model m1 generally performed better than m2 for "Baron - 1 core"
  - Baron 1 core was almost the same as 4 and 16 core execution
  - Octeract with model m1 and m2 together (i.e. taking minimum of the result after using both m1 and m2 with Octeract)
    - 1 core was comparable with 4 and 16 core execution in general
    - 1 core was $\approx$ 27% of the times better as compared to 4 and 16 cores
    - 16 cores was $\approx$ 18% of the times better as compared to 1 and 4 cores
- Execution time limit of 4 hours
  - Octeract with model m1 and m2 together
    - 16 cores was $\approx$ 63% times better as compared to 1 and 4 cores
    - 1 core was $\approx$ 27% times better as compared to 16 cores
    - 4 cores was $\approx$ 9% times better as compared to 16 cores
  - Baron with model m1 and m2 together
    - 4 cores was better than 1 and 16 cores execution
    - 1 core was quite comparable with 4 cores execution
      - After searching, it was found that BARON uses multiple cores only when there are integer variables in the problem. Additionally, it does so only during the lower bounding step of the algorithm by launching CPLEX or CBC in parallel mode.
- Among all combinations of "Octeract and Baron", "models m1 and m2" and "1, 4 and 16 core" with time limit of 5 minutes, using "Octeract and Baron" together with both "models m1 and m2" with 1 core turned out to be the best or they had a solution which was at most 1% poor than the overall best among "$2 * 2 * 3 = 12$" combinations (i.e. in case of minimization of an objective function, if the best result is 100 then 1% poor result

would be 101). So, the final result would be:

$$\min \begin{pmatrix} \text{CPU=1 core} \\ \text{Time=5 minutes} \\ \text{Models=[m1, m2]} \\ \text{Solvers=[Octeract, Baron]} \end{pmatrix}$$

- For best result across all combinations with models m1 and m2, use:

$$\min \begin{pmatrix} \text{CPU=1 core} \\ \text{Time=[5 minutes, 4 hours]} \\ \text{Models=[m1, m2]} \\ \text{Solvers=[Octeract, Baron]} \end{pmatrix}$$

  - This results in best results in $\approx$ 81% data files, and a just $\approx$ 1% poor result (i.e. in case of minimization of an objective function, if the best result is 100 then 1% poor result would be 101) in other $\approx$ 19% data files.

Chapter 3

Integration of JalTantra Website and Solvers

3.1 JalTantra Frontend - Overview

Figure 3.1 gives a high-level overview of the frontend of the JalTantra website. The end user has to enter the data related to their pipeline network either manually or load it from an existing file (XML file, Excel file or Branch file). Once the network data has been entered or loaded on the website, the user has to just click the "Optimize" button to submit a request to the backend to optimize the network.

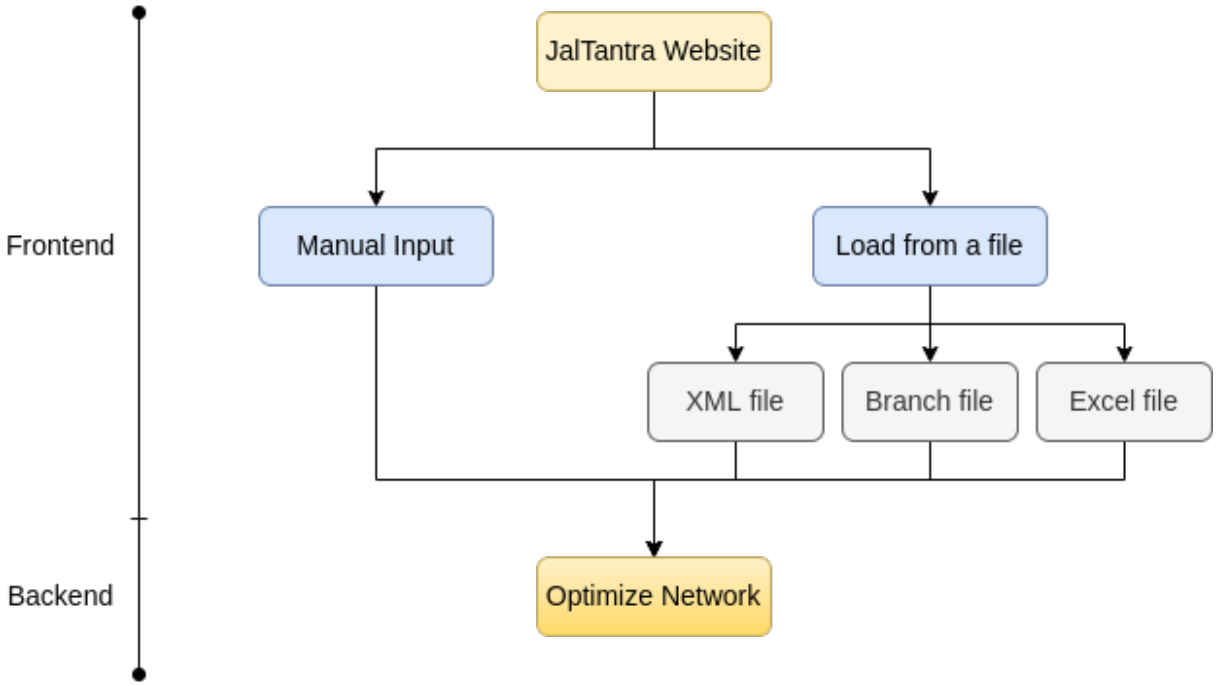

Figure 3.1: High-level overview of the JalTantra frontend

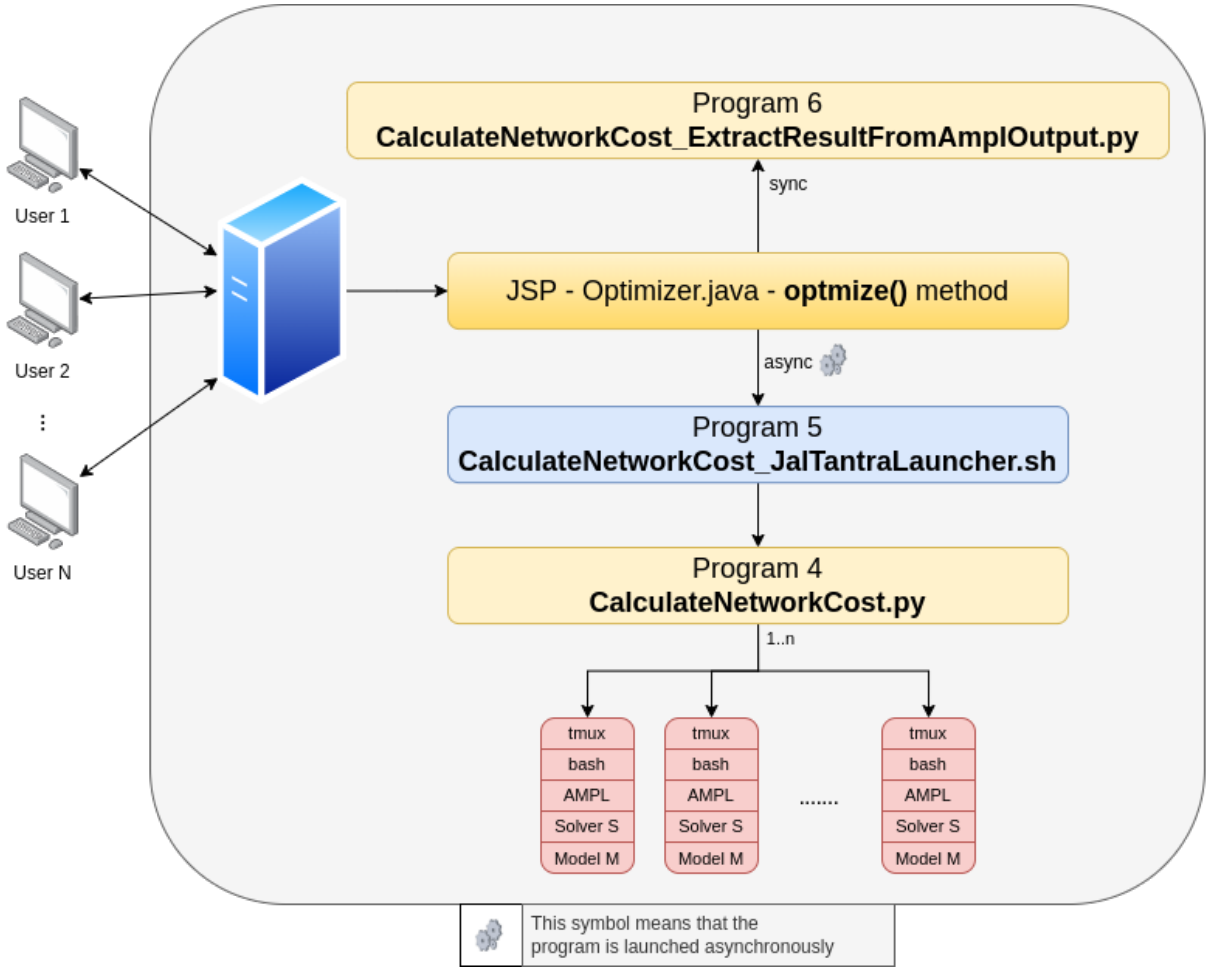

Figure 3.2: High-level overview of the JalTantra backend

3.2 JalTantra Backend - Overview

Figure 3.2 highlights the key components of the JalTantra backend and shows how they interact with each other.

3.3 JalTantra Backend - Core Components

Before giving the details, I would like to explain the directory structure of the project, the programs that are used, and the log files that would be generated.

Paths - Content (files and directories) under `/path/to/ScriptsRootDir` (i.e. the directory which would have all the data related to all the requests that were made from the JalTantra website). The same directory structure is maintained by the project JalTantra-Code-and-Scripts [8].

Note that triangle brackets denote dynamic content (i.e. the content that changes).

1. Files (dir)

   (a) Model (dir)

   (b) Data (dir)

      i. m1_m2 (dir)

      ii. m3_m4 (dir)

2. DataNetworkGraphInput (dir)

3. DataNetworkGraphInput_hashed (dir)

   (a) <DataFileHash>.R (network file)

   (b) <DataFileHash>.R.log (log file)

4. NetworkResults (dir)

   (a) SolutionData (dir)

      i. Octeract file with name regex "`at[0-9]+\.octsol`" (file)

      ii. Baron directories with name regex "`baron_tmp[0-9]+`" (dir)

         A. amplmodel.bar (file)

         B. dictionary.txt (file)

         C. res.lst (file)

         D. sum.lst (file)

         E. tim.lst (file)

   (b) <DataFileHash> (dir)

      i. <Solver>_<ModelShortForm>_<DataFileHash> (dir)

         A. std_out_err.txt (file)

      ii. 0_graph_network_data_testcase.R (hardlink)

      iii. 0_metadata (file)

      iv. 0_status (file)

      v. 0_result.txt (file)

      vi. 0_result_summary.txt (file)

Programs (from project JalTantra-Code-and-Scripts [8])

1. htop_monitor_writer.sh

2. htop_monitor_reader.sh

3. Output table extractors

   (a) Baron

      i. output_table_extractor_baron.sh

   (b) Octeract

      i. output_table_extractor_octeract.sh

4. CalculateNetworkCost.py
5. CalculateNetworkCost_JaltantraLauncher.sh
6. CalculateNetworkCost_ExtractResultFromAmplOutput.py

Logs (paths are relative to `/path/to/ScriptsRootDir`)

1. log_jaltantra_CalculateNetworkCost_JaltantraLauncher.log
   - Logs in this files are written by "Program 5"
2. DataNetworkGraphInput_hashed/<DataFileHash>.R.log
   - It is configured in "Program 5" that output (stdout and stderr) of "Program 4" should go to this file
3. NetworkResults/<DataFileHash>/<Solver>_<ModelShortForm>_<DataFileHash> /std_out_err.txt
   - It is configured in "Program 4" that output (stdout and stderr) of solvers should be written to this file
4. Tomcat server logs

3.4 JalTantra Backend - Java Server Pages (JSP)

Refer Figure 3.3 for the upcoming explanation. Once the user clicks on the "Optimize" button, the data from the frontend is converted into JavaScript Object Notation (JSON) and then sent to the backend where the `doPost(...)` method receives it. The JSON data is parsed using a Java library and stored in Java objects. Then, `optimize(...)` method is called which does the true work on the network submitted by the user. Firstly, few validation checks are performed on the user submitted network. If everything turns out good, then we proceed with creating a network file at the server (which would be used by the solvers). Using the SHA-256 hash of the network file created, we check whether the network was submitted in

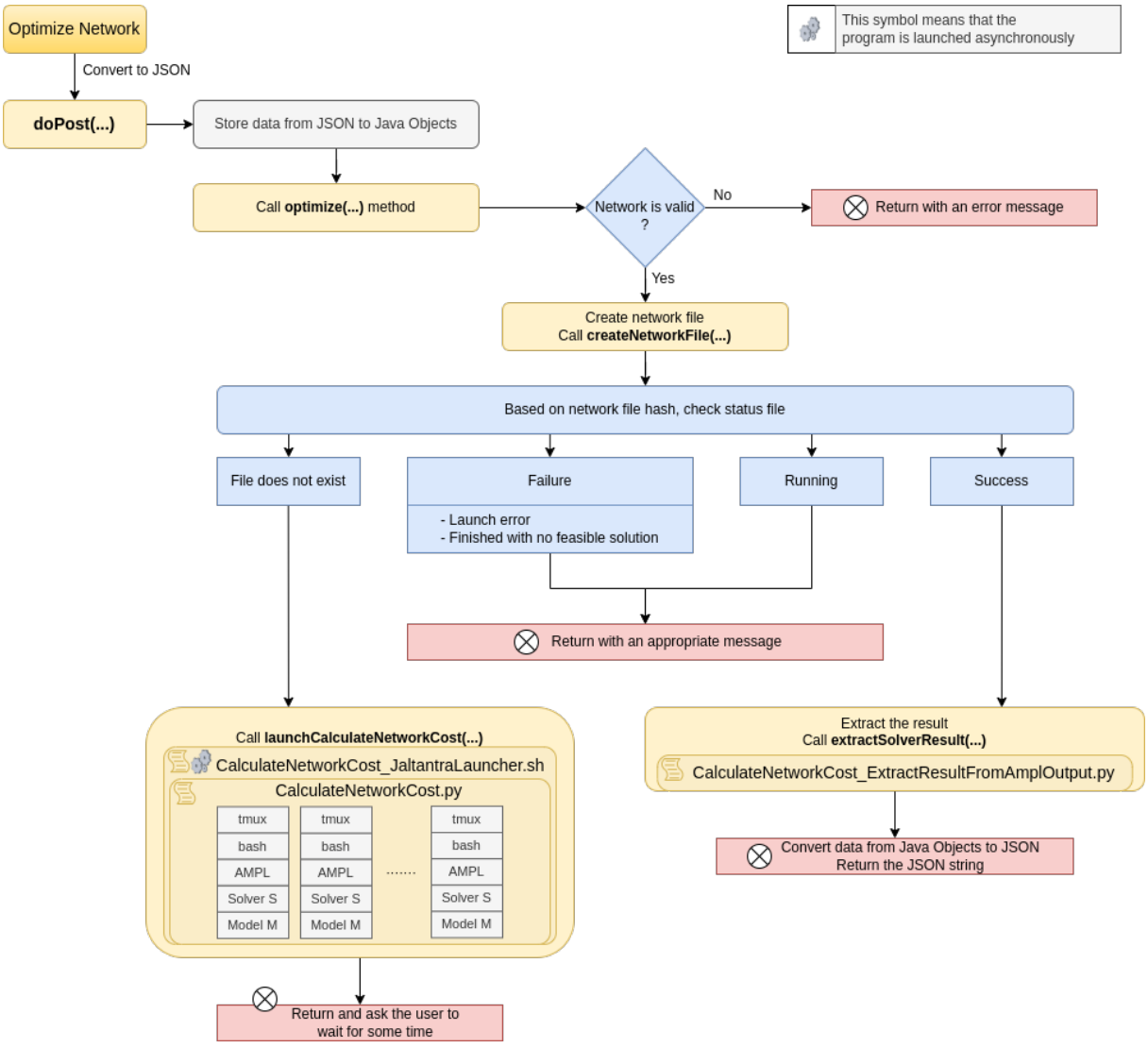

Figure 3.3: High-level overview of the JalTantra backend JSP code

the past or not, and check whether the network was already optimized or not. If the network is seen for the first time (i.e. status file does not exist), then we asynchronously launch `CalculateNetworkCost_JaltantraLauncher.sh` which in turn gives the network file to `CalculateNetworkCost.py` to launch solvers to get the best result for the network, and while the solvers are running we send reply to the user that they would have to wait for some time. But, if the solvers "are already running" or "failed due to some reason" for the network file that was submitted by the user, then according to the value of the status file we either ask the user to wait as the solvers are still working to find the solution for the network, or inform them that the optimization operation failed (along with the possible reason for the failure). But, if the network was optimized in the past, then we directly extract the solution from the past optimization outcome and return it to the user.

Following are the detailed steps that are performed once the `optimize()` method (refer Figure 3.3) is called:

1. Validate the request
   (a) Head should be greater than or equals to elevation
   (b) Network should be connected
2. Create data file (which is input to AMPL and solvers)
   (a) Generate a unique file name based on time and a random number
   (b) Write the request's data in AMPL's required format to the above file in the directory at "Path 2".
   (c) Find SHA-256 hash of the file and move the file to the directory at "Path 3" with new file name as "<DataFileHash>.R" (i.e. "Path 3.a")
   (d) If any problem (i.e. exception) occurs in the above steps, then we perform the necessary clean up operations and return failure message to the frontend
3. **If** file at "Path 3.a" was newly created **or** file at "Path 4.b.iv" does not exist (it is assumed that no manual fiddling is done with directories at "Path 3 and 4") – then asynchronously execute "Program 5" and ask the user at the frontend to wait.
   Else go to step 4
4. // Request to solve the network was already launched in the past. It may be running or may have finished.
   If status file at "Path 4.b.iv" says that the network was successfully optimized, then go to step 5
   Else go to step 6

5. We use "Program 6" to extract the results from the output file at "Path 4.b.i.A" based on the solver-model combination which gave the best result (this information is present in 4.b.v) among the executed combinations, and present it to the user.

6. Depending on the content of the status file at "Path 4.b.iv" we inform the user that the solvers are running, or solvers failed to serve the request.

3.5 JalTantra Backend - Core Solver Managing Program

In this section I will talk about the `CalculateNetworkCost.py` (i.e. Program 4) that does the true work of launching solvers for different model files in parallel, constantly monitoring them for errors, stopping them if time limit is crossed, and storing information about the best solution among the launched solver-model combinations.

In Figure 3.4, the first part of the three parts of the working of `CalculateNetworkCost.py` is shown. The program is launched using command line interface (CLI) where all the necessary configuration parameters are set (one can get the CLI help text by executing `CalculateNetworkCost.py --help`). The program starts by validating the CLI parameters received and stores them in a Python object after parsing them. The hash of the passed network file is also calculated and stored. Before beginning the true work, the necessary directories are created, a hard link to the network file (that is passed via the CLI) is created at "Path 4.b.ii", and metadata about the request is stored at "Path 4.b.iii".

We divide the solver-model combinations into two batches. The first batch has `--jobs` number of combinations and the remaining combinations are put in the second batch. This is done so that at max `--jobs` number of parallel instances of solver-model combinations are running. Configuring this parameter will help decide the max number of users that the server can handle without any regression on the quality of results. For example, for a 48 core server and `--jobs` value to be 4, the server can serve $48/4 = 12$ users in parallel without hurting the quality of results. However, if more users use the JalTantra website at the same time, then the solver-model instances will compete for CPU resources. But, the server will continue to return the results to the user on timely bases because the solver-model instances will continue to get same wall-clock execution time which is server defined (let us call this `ServerDefinedExecutionTimeLimitForSolvers` to be T), but the true CPU execution time for the solver instances will be equal to:

$= T * min(1, (48/MaxParallelJobs)/UserCount$

So, for $T = 5\ minutes$, $MaxParallelJobs = 4$ and $UserCount = 18$, the solver-model combination will get true CPU execution time of

$= 5 * min(1, (48/4)/18) = 5 * min(1, 12/18) = 5 * min(1, 2/3) = 5 * 2/3 \approx 3.33 minutes$

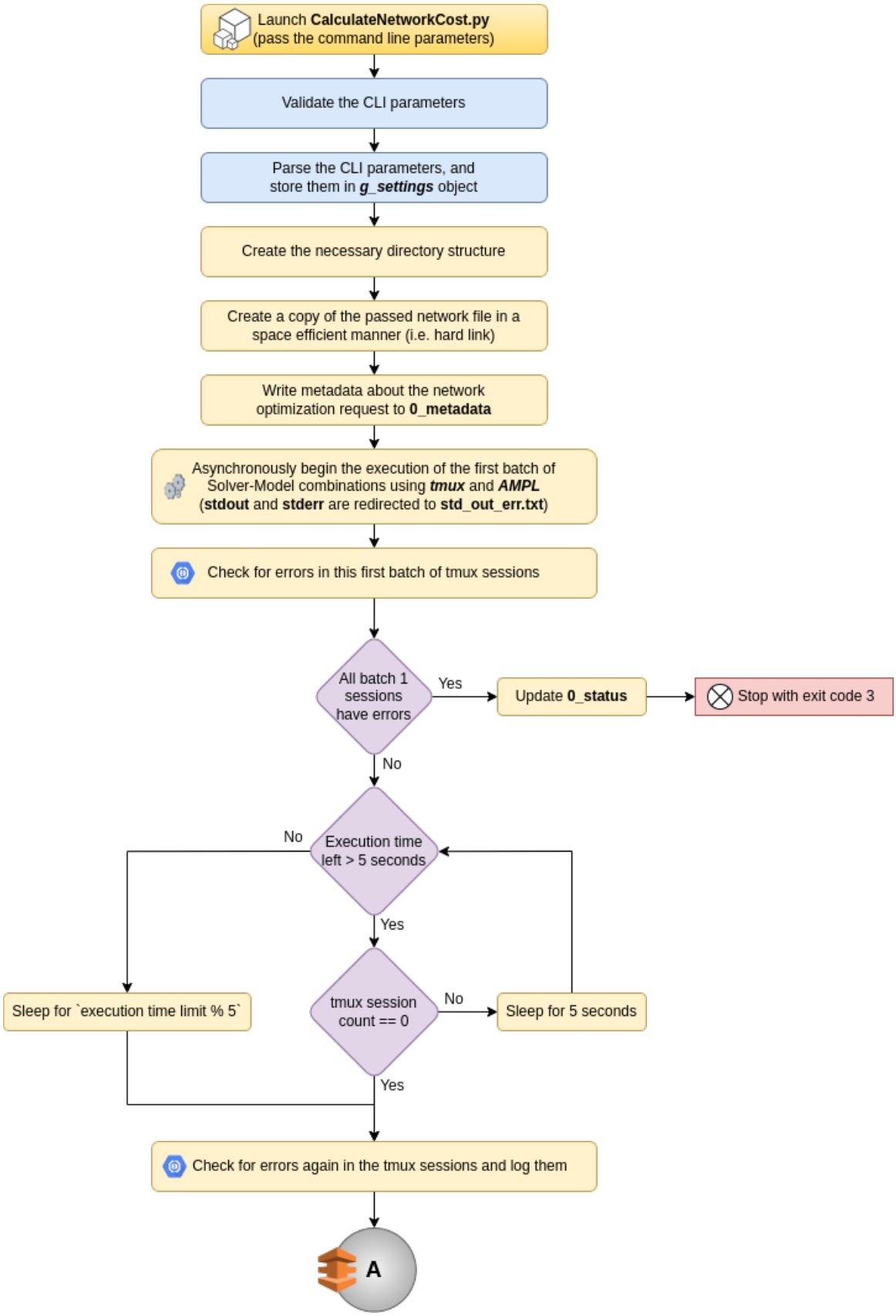

Figure 3.4: Part 1 of 3 - High-level overview of the Python program which interacts with the solvers to find the best result

Using `tmux`, we asynchronously start all solver-model combinations of the first batch, and check for errors by parsing their output which is stored at "Path 4.b.i.A". If all `tmux` sessions of the first batch terminate too early, then we report the solver launch error to the status file (at "Path 4.b.iv") and stop the program. If one or more `tmux` sessions are running then we wait for them to either stop on their own or cross the time limit T. Once we are done with this waiting, we again check for errors in this first batch of `tmux` sessions and log them.

Now, we see if anyone of the `tmux` session from the first batch has found a feasible solution or not. If not, then we give them extra time (at max 300 seconds) to find a feasible solution. After that, we prepare for the second batch of solver-model combinations. While there is a solver-model combination in batch two that is not yet launched, we constantly monitor the running `tmux` sessions and check whether they have reached their execution time limit or not. Once a running session reaches its time limit, we stop it and start a pending solver-model combination from the second batch. After all solver-model combinations from batch two have been launched, we just wait for the running sessions to end, or once they reach their time limit, we stop them by sending `SIGINT` (i.e `CTRL+C`) to the solvers so that they start the wind-up process, and we finally check for errors again and log them.

Now, though we have asked the solvers to stop solving the network, they do not immediately stop, instead they do some shutdown actions (e.g. connecting to their servers for communicating the results) and then stop. So, this shutdown procedure may take time. Hence, we wait for the tmux sessions to end before proceeding. After that, we copy the solver generated solution files from `/tmp` to "Path 4.a".

Finally, we extract the solutions of all solver-model combinations that we had run and write the details of the best solution to "Path 4.b.v", and the summary is written to "Path 4.b.vi". Lastly, the status file at "Path 4.b.iv" is updated and the program ends.

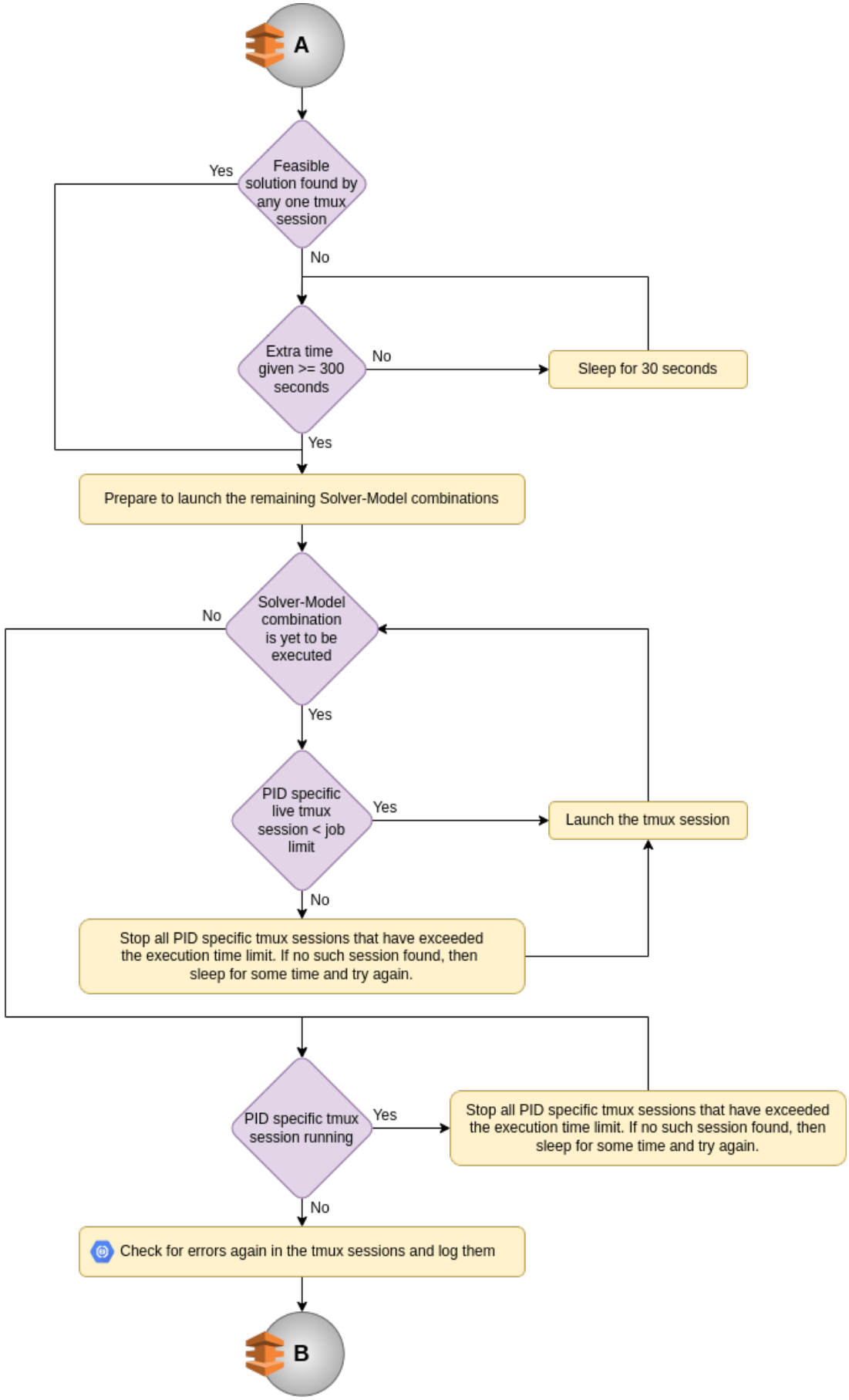

Figure 3.5: Part 2 of 3 - High-level overview of the Python program which interacts with the solvers to find the best result

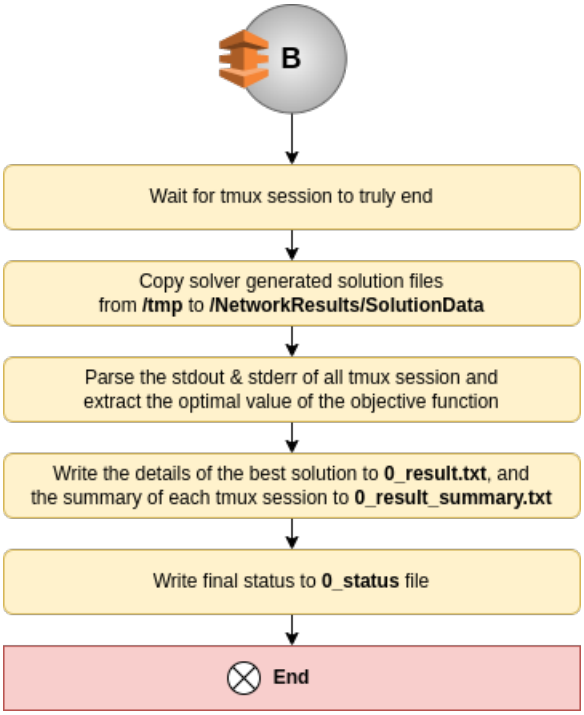

Figure 3.6: Part 3 of 3 - High-level overview of the Python program which interacts with the solvers to find the best result

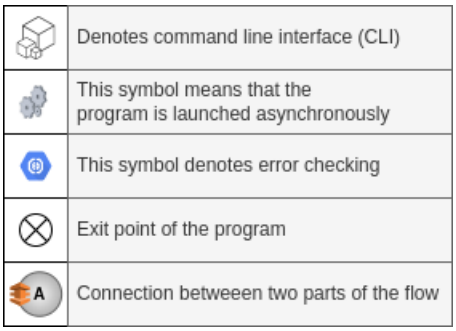

Figure 3.7: Symbol table for Figure 3.4, Figure 3.5, Figure 3.6

Chapter 4

Challenges Faced

- The first challenge was understanding the format in which the solvers write logs and show progress of their work. So, I had to spend a good amount of time experimenting how the solvers behaved in different situations. Due to lack of proper documentation, I had also contacted the company which developed the solver to help me understand the output format.
- The second challenge was to use `bash` commands from Python. Initially it seemed very easy. But, when the solver analysis program which was mentioned in "Section 2.2" was running, it was found that bash commands did not run as expected from Python. So, after lots of reading and trial and error it was found that the cause of this issue was the shell that is being used to parse the commands that we are calling. By default, `sh` and `dash` shell are used. But, they lack the functionalities that `bash` provides. Hence, I had to explicitly use `bash` for running the shell commands from Python.
- The third challenge was to ensure that commands executed on the server continue to run even after the `ssh` connection to the server is over, and to also have the ability to view its output and interact with the program as though we had kept the connection open. A terminal multiplexer called `tmux` came to the rescue. It allowed us to attach and detach from terminals while keeping the commands that are running in those terminals to keep running as they are.
- The fourth challenge was launching the running solvers in the background and managing them. It would be complicated had only Python functionalities been used. But, thanks to `tmux` it was possible to launch the solvers in the background and keep track of them without much issue.
- The fifth challenge was that solvers take a few minutes to find a feasible solution and it is possible that the end-users internet connection could break before the results are obtained

by them from the backend. So, asynchronous request submission and result retrieval was required. Here, the concept of collision resistant hashing (e.g. SHA-256) made it possible. We would launch "Program 4" in the background and have it write the results to a file on the server. Now, whenever the end-user returns and requests for the results, we just check if the request was submitted in the past or not, and accordingly reply back to them.

Chapter 5

Key Changes Introduced

- Improved the directory structure of data file and model files
- Consistence was introduced in Data and Model file naming
- Fixed bugs of model file "m2_basic2"
- Shifted from old development softwares and IDE's to better and newer tools.
  - Shifted from Eclipse to JetBrains' IDEs
  - Introduced GitKraken to help anyone easily take advantage of git and GitHub
- Implemented the core backend of JalTantra from scratch that interacts with AMPL and the solvers. Various options are provided for configurability of the backend, some of them are:
  * Can be used to generate data to do "analysis and evaluate" solvers and models. For example, which solvers are better performing when executed for long time (e.g. 4 hours) and short time (e.g. 5 minutes), which model gives better results when used to optimize networks in long run (e.g. 4 hours) and short run (e.g. 5 minutes)
  * `--solver-models` option to execute execute multiple solver model combination on the same network file and then get the best result among them
  * `--time` option to control the execution time limit of the solvers
  * `--threads-per-solver-instance` option to set the number of threads a solver instance can consume when optimizing a network file
  * `--jobs` option available for the backend to decide upon ideal max load that we want the server to handle

    * Built with modularity to easily add new solvers to the JalTantra backend, thus expanding the scope and effectiveness of the JalTantra project.

- The JalTantra website changes:
  - Optimization results can be obtained even if connection to the server is lost
  - Efficient use of resources by avoiding repeated computation if same network is submitted by multiple users at the same time for optimization
  - Optimization requests are cached so that:
    * The end user gets the results immediately without having to wait for the backend to solve the network again
    * It prevents repeated use of computational power on the same network
  - Improved UX of the website. Now, "Load/Save Files > Load File" and "Load/Save Files > Save Input File" expand with a single click. Previously, they had to be double clicked, and consistency lacked because other components would expand with a single click only.
  - Previously, the JalTantra website code was hosted on Dropbox [11]. Now, the JalTantra website code is privately hosted on GitHub [12] for
    * easy collaboration among multiple people
    * better progress tracking with the help of git commits
    * centralized tracking of bugs fixes and feature additions using GitHub issues

    Previously, it was not possible for multiple people to work on the same project, bug fixes and feature additions had to be manually tracked, and it was not possible for people to do discussions like that possible on GitHub issues.
- Verbose logging has been done across the JalTantra backend to find the cause of any issue regarding network optimization within few minutes.
- Documented guidelines for development environment setup
- Documented deployment steps for easy of everyone who works on this project

Chapter 6

Results and Comparison

Following are the results generated by the newly developed system on five sample input files:

- For input test case file "two_loop.xls"
  - Cyclic network - it has 7 nodes, 8 links and 14 commercial pipes
  - Gave total establishment cost as 403,606.76 after 170 seconds
- For input test case file "HG_SP_1_1.xls"
  - Cyclic network - it has 73 nodes, 101 links and 12 commercial pipes
  - Gave total establishment cost as 3,097,228.26 after 183 seconds
- For input test case file "HG_SP_1_2.xls"
  - Cyclic network - it has 73 nodes, 101 links and 12 commercial pipes
  - Gave total establishment cost as 3,391,130.10 after 133 seconds
- For input test case file "HG_SP_1_3.xls"
  - Cyclic network - it has 73 nodes, 101 links and 12 commercial pipes
  - Gave total establishment cost as 5,947,774.76 after 133 seconds
- For input test case file "HG_SP_1_4.xls"
  - Cyclic network - it has 78 nodes, 100 links and 12 commercial pipes
  - Gave total establishment cost as 3,554,911.75 after 133 seconds

Chapter 7

Conclusion

- I have successfully done analysis of combinations of solvers, models and the CPU cores that are used for solving the network, and presented it in "Section 2.3".
- I have done complete end-to-end integration of the JalTantra website with global solvers (like Octeract and Baron), and deployed my work on the loop server (i.e. the server with IIT-B Local Network IP *10.129.6.131*). Currently my work is deployed at `http://10.129.6.131:8080/jaltantra_loop_dev_v1/` and in near future it will be accessible at `http://10.129.6.131:8080/jaltantra_loop/`.
- I have also compared and shown that "the quality of results" and "the quickness" of the newly developed system (i.e my work) is way better as compared to the previously developed and deployed systems.
- All the code is present at private GitHub repositories [8] and [12].

Appendix A

Solver Model Execution Results

Note that N/A denotes that no feasible solution was found by the solver, and TLE denotes too long execution (i.e. if we wanted the best result under 300 seconds and the first solution given by the solver took 350 seconds, then TLE is displayed)

Table A.1: One CPU core per solver instance

		Cores	1	1	1	1
		Solver	Baron	Baron	Octeract	Octeract
		Time (seconds)	0:05:00	4:00:00	0:05:00	4:00:00
Data files	Model file					
d1	m1		403607	403607	4.17E+05	4.17E+05
d1	m2		403607	403607	4.17E+05	4.17E+05
d1	m3		403607	403607	4.37E+05	4.37E+05
d1	m4		403607	403607	4.17E+05	4.17E+05
d2	m1		6.06E+06	6.06E+06	6.32E+06	6.08E+06
d2	m2		6.12E+06	6.06E+06	6.07E+06	6.07E+06
d2	m3		6.07E+06	6.06E+06	3.11E+07	8.66E+06
d2	m4		6.74E+06	6.06E+06	6.06E+06	6.06E+06
d3	m1		1.21E+07	1.21E+07	6.32E+07	1.29E+07
d3	m2		1.91E+07	1.24E+07	6.32E+07	6.32E+07
d3	m3		1.21E+07	1.21E+07	6.32E+07	6.32E+07
d3	m4		1.22E+07	1.22E+07	1.22E+07	1.22E+07
d4	m1		1.84E+07	1.84E+07	9.55E+07	9.55E+07
d4	m2		1.92E+07	1.92E+07	1.84E+07	1.84E+07
d4	m3		1.88E+07	1.87E+07	9.55E+07	9.55E+07
d4	m4		1.93E+07	1.93E+07	1.86E+07	1.86E+07
d5	m1		1.00E+51	7.99E+07	9.25E+06	8.95E+06
d5	m2		8.85E+06	8.82E+06	9.05E+06	9.05E+06
d5	m3		N/A	N/A	N/A	N/A
d5	m4		N/A	N/A	N/A	N/A
d6	m1		3.55E+06	3.55E+06	3.80E+06	3.80E+06
d6	m2		4.25E+06	4.25E+06	3.56E+06	3.56E+06
d6	m3		N/A	N/A	N/A	N/A
d6	m4		3.78E+06	3.78E+06	9.05E+06	9.05E+06
d7	m1		1.00E+51	1.00E+51	8.66E+07	4.91E+06
d7	m2		1.00E+51	2.11E+07	3.40E+06	3.40E+06
d7	m3		N/A	N/A	N/A	N/A
d7	m4		3.93E+06	3.93E+06	1.02E+07	1.02E+07
d8	m1		6.92E+06	6.92E+06	1.20E+07	1.20E+07
d8	m2		6.96E+06	6.96E+06	7.02E+06	7.02E+06
d8	m3		N/A	N/A	N/A	N/A
d8	m4		6.92E+06	6.92E+06	1.11E+07	1.11E+07
d9	m1		1.00E+51	1.00E+51	1.33E+08	1.33E+08
d9	m2		1.00E+51	3.23E+07	1.68E+07	9.32E+06
d9	m3		N/A	N/A	N/A	N/A
d9	m4		TLE	9.06E+06	1.33E+08	1.73E+07
d10	m1		9.33E+06	9.33E+06	1.60E+08	1.60E+08
d10	m2		1.61E+07	1.61E+07	9.50E+06	9.50E+06
d10	m3		N/A	N/A	N/A	N/A
d10	m4		1.07E+07	1.07E+07	1.60E+08	1.56E+07
d11	m1		1.07E+07	1.07E+07	1.32E+08	1.32E+08
d11	m2		1.00E+51	1.00E+51	1.17E+07	1.17E+07
d11	m3		N/A	N/A	N/A	N/A
d11	m4		1.00E+51	1.00E+51	1.32E+08	1.32E+08

Table A.2: Four CPU cores per solver instance

		Cores	4	4	4	4
		Solver	Baron	Baron	Octeract	Octeract
		Time (seconds)	0:05:00	4:00:00	0:05:00	4:00:00
Data files	Model file					
d1	m1		403607	403607	4.04E+05	4.04E+05
d1	m2		403607	403607	4.17E+05	4.17E+05
d1	m3		403607	403607	4.88E+05	4.37E+05
d1	m4		403607	403607	4.17E+05	4.17E+05
d2	m1		6.06E+06	6.06E+06	6.06E+06	6.06E+06
d2	m2		6.12E+06	6.06E+06	6.11E+06	6.11E+06
d2	m3		6.07E+06	6.06E+06	3.11E+07	3.11E+07
d2	m4		1.00E+51	6.06E+06	6.06E+06	6.06E+06
d3	m1		1.21E+07	1.21E+07	6.32E+07	1.23E+07
d3	m2		1.91E+07	1.25E+07	6.32E+07	6.32E+07
d3	m3		1.21E+07	1.21E+07	6.32E+07	6.32E+07
d3	m4		1.22E+07	1.22E+07	1.23E+07	1.23E+07
d4	m1		1.84E+07	1.84E+07	9.55E+07	9.55E+07
d4	m2		1.92E+07	1.92E+07	1.84E+07	1.84E+07
d4	m3		1.88E+07	1.87E+07	9.55E+07	9.55E+07
d4	m4		1.93E+07	1.93E+07	2.45E+07	2.45E+07
d5	m1		1.00E+51	7.99E+07	9.01E+06	9.01E+06
d5	m2		8.85E+06	8.82E+06	9.05E+06	9.05E+06
d5	m3		N/A	N/A	N/A	N/A
d5	m4		N/A	N/A	N/A	N/A
d6	m1		3.55E+06	3.55E+06	3.84E+06	3.84E+06
d6	m2		4.25E+06	4.25E+06	3.56E+06	3.56E+06
d6	m3		N/A	N/A	N/A	N/A
d6	m4		3.78E+06	3.78E+06	8.61E+06	8.61E+06
d7	m1		1.00E+51	2.11E+07	8.66E+07	1.10E+07
d7	m2		1.00E+51	2.11E+07	3.41E+06	3.40E+06
d7	m3		N/A	N/A	N/A	N/A
d7	m4		3.93E+06	3.93E+06	9.62E+06	9.62E+06
d8	m1		6.92E+06	6.92E+06	1.14E+07	1.14E+07
d8	m2		6.96E+06	6.96E+06	7.02E+06	7.02E+06
d8	m3		N/A	N/A	N/A	N/A
d8	m4		6.92E+06	6.92E+06	1.11E+07	1.11E+07
d9	m1		1.00E+51	3.23E+07	1.91E+07	1.91E+07
d9	m2		1.00E+51	1.00E+51	1.79E+07	1.79E+07
d9	m3		N/A	N/A	N/A	N/A
d9	m4		TLE	9.06E+06	1.33E+08	1.33E+08
d10	m1		9.33E+06	9.32E+06	1.60E+08	9.42E+06
d10	m2		1.61E+07	1.61E+07	1.04E+07	9.75E+06
d10	m3		N/A	N/A	N/A	N/A
d10	m4		1.07E+07	1.07E+07	1.60E+08	1.60E+08
d11	m1		1.07E+07	1.07E+07	1.32E+08	2.00E+07
d11	m2		1.00E+51	3.21E+07	1.82E+07	1.82E+07
d11	m3		N/A	N/A	N/A	N/A
d11	m4		1.00E+51	1.00E+51	1.32E+08	1.32E+08

Table A.3: 16 CPU cores per solver instance

		Cores	16	16	16	16
		Solver	Baron	Baron	Octeract	Octeract
		Time (seconds)	0:05:00	4:00:00	0:05:00	4:00:00
Data files	Model file					
d1	m1		403607	403607	4.04E+05	4.04E+05
d1	m2		403607	403607	4.17E+05	4.17E+05
d1	m3		403607	403607	4.74E+05	4.40E+05
d1	m4		403607	403607	4.20E+05	4.20E+05
d2	m1		6.06E+06	6.06E+06	6.06E+06	6.06E+06
d2	m2		6.12E+06	6.06E+06	6.06E+06	6.06E+06
d2	m3		6.07E+06	6.06E+06	3.11E+07	8.67E+06
d2	m4		1.00E+51	6.06E+06	6.06E+06	6.06E+06
d3	m1		1.21E+07	1.21E+07	1.30E+07	1.24E+07
d3	m2		1.91E+07	1.24E+07	1.21E+07	1.21E+07
d3	m3		1.21E+07	1.21E+07	6.32E+07	6.32E+07
d3	m4		1.22E+07	1.22E+07	1.23E+07	1.23E+07
d4	m1		1.84E+07	1.84E+07	9.55E+07	2.39E+07
d4	m2		1.92E+07	1.92E+07	1.84E+07	1.84E+07
d4	m3		1.88E+07	1.88E+07	9.55E+07	9.55E+07
d4	m4		1.93E+07	1.93E+07	2.45E+07	2.45E+07
d5	m1		1.00E+51	7.99E+07	9.06E+06	9.02E+06
d5	m2		8.85E+06	8.82E+06	9.05E+06	9.05E+06
d5	m3		N/A	N/A	N/A	N/A
d5	m4		N/A	N/A	N/A	N/A
d6	m1		3.55E+06	3.55E+06	3.57E+06	3.57E+06
d6	m2		4.25E+06	4.25E+06	3.56E+06	3.56E+06
d6	m3		N/A	N/A	N/A	N/A
d6	m4		3.78E+06	3.78E+06	8.61E+06	8.61E+06
d7	m1		1.00E+51	2.11E+07	1.60E+07	1.60E+07
d7	m2		1.00E+51	1.00E+51	3.40E+06	3.40E+06
d7	m3		N/A	N/A	N/A	N/A
d7	m4		3.93E+06	3.93E+06	9.62E+06	9.62E+06
d8	m1		6.92E+06	6.92E+06	1.14E+07	1.14E+07
d8	m2		6.96E+06	6.96E+06	7.02E+06	7.02E+06
d8	m3		N/A	N/A	N/A	N/A
d8	m4		6.92E+06	6.92E+06	1.11E+07	1.11E+07
d9	m1		1.00E+51	3.23E+07	1.91E+07	1.91E+07
d9	m2		1.00E+51	1.00E+51	1.71E+07	1.71E+07
d9	m3		N/A	N/A	N/A	N/A
d9	m4		TLE	9.06E+06	1.33E+08	1.33E+08
d10	m1		9.41E+06	9.41E+06	1.60E+08	9.96E+06
d10	m2		1.61E+07	1.61E+07	1.76E+07	1.05E+07
d10	m3		N/A	N/A	N/A	N/A
d10	m4		1.07E+07	1.07E+07	1.60E+08	1.60E+08
d11	m1		1.07E+07	1.07E+07	1.32E+08	1.15E+07
d11	m2		1.00E+51	1.00E+51	1.84E+07	1.84E+07
d11	m3		N/A	N/A	N/A	N/A
d11	m4		1.00E+51	1.00E+51	1.32E+08	1.32E+08

Acknowledgements

I am thankful to **Prof. Om Damani** for providing me with the opportunity to work on this project. I am grateful to him for guiding me on the project and giving me his valuable feedback.

Fenil Gaurang Mehta
IIT Bombay
24 June 2022

References

[1] Nikhil Hooda and Om Damani. Jaltantra: A system for the design and optimization of rural piped water networks. *INFORMS Journal on Applied Analytics*, 49, 11 2019.

[2] Saumya Goyal, Om Damani, and Ashutosh Mahajan. Cost optimization of water distribution networks: Model refinement is better than problem-specific solving techniques, 2021.

[3] N. Hooda and O. Damani. A system for optimal design of pressure constrained branched piped water networks. *Procedia Engineering*, 186:349–356, 2017. XVIII International Conference on Water Distribution Systems, WDSA2016.

[4] Wikipedia - Solver. https://en.wikipedia.org/wiki/Solver.

[5] Wikipedia - Octeract. https://en.wikipedia.org/wiki/Octeract_Engine.

[6] Tutorial of AMPL for Linear Programming. https://www.cs.uic.edu/~hjin/files/ampl_tutorial.pdf.

[7] AMPL Handout - Introduction to AMPL. https://www.math.wsu.edu/faculty/bkrishna/FilesMath567/S17/Handouts/Handout_AMPL_1.pdf.

[8] Source code of JalTantra backend components. https://github.com/fenilgmehta/JalTantra-Code-and-Scripts.

[9] Execution Results of Solver-Model Combinations for Analysis. https://docs.google.com/spreadsheets/d/1pZNZ-Vjvj0xc-h9pVS85OSNSdkC2940c/edit?usp=sharing&ouid=101940378676875078716&rtpof=true&sd=true.

[10] Enrico Creaco and Marco Franchini. The identification of loops in water distribution networks. *Procedia Engineering*, 119:506–515, 2015. Computing and Control for the Water Industry (CCWI2015) Sharing the best practice in water management.

[11] Source code shared by Saumya Goyal. https://www.dropbox.com/sh/2x4wr2vccqmnh9e/AAALBLx-ilpKyUe5_AM7LqMAa?dl=0.

[12] Source code of JalTantra website. https://github.com/fenilgmehta/JalTantra-Website.